ternet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

THE TEST SETUP · PUMP FAULT DETECTION ON ESP32-S3

SIEMENS

Two pump families, four units, two mounting surfaces



Wilo TO3 (green) and KSB Etaline TO2 (red) on the rig.

	Wilo	KSB Etaline
Type	Vertical inline centrifugal, motor directly above the casing	Vertical inline centrifugal, motor directly above the casing
Units on the rig	Wilo TO3, Wilo TO4	Etaline TO1, Etaline TO2

THE TWO MOUNTING SURFACES

Every unit was measured on a concrete foundation and metal plate.

2
PUMP FAMILIES
Wilo · KSB Etaline

4
UNITS ON THE RIG
TO1 · TO2 · TO3 · TO4

2
MOUNTING SURFACES
Concrete → metal plate

21
MEASUREMENT DAYS
Messtag 01 – 21, Mar–Apr 2023

Both axes are context, not noise: pump family changes casing stiffness and hydraulic design; the mounting surface changes how vibration is transmitted and damped. The same fault shows up at a different amplitude depending on the unit and the surface under it.

Notes

ENG
IN

Transitions Animations Slide Show Record Review View Help
Present in Teams
the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

WHAT WE MEASURED, AND WHAT WE KEPT

Four fault states were run — the model answers three

SIEMENS

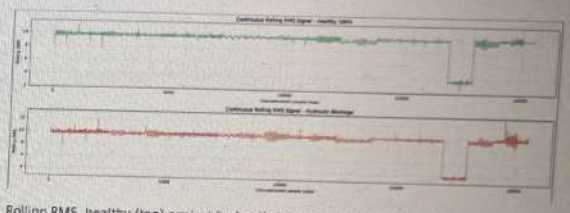
THE FOUR STATES THAT WERE RUN

Healthy	IN THE MODEL	Pump at its normal duty point.
Cavitation	IN THE MODEL	Vapour bubbles form and collapse.
Suction blockage	IN THE MODEL	Restriction at the pump inlet.
Hydraulic blockage	DROPPED	Restriction on the discharge side.

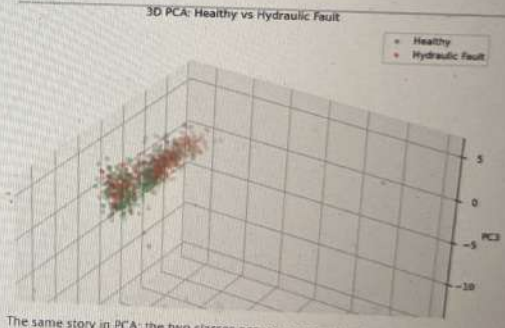
WHY HYDRAULIC BLOCKAGE WAS DROPPED

In the accelerometer signal, hydraulic blockage and healthy operation look almost identical — the plots opposite are the evidence. Vibration alone cannot separate them, and forcing it in as a fourth class costs accuracy on the three that do separate cleanly.

It is not gone for good: once temperature and other channels join the accelerometer, the missing signal should be there and hydraulic blockage goes back in.



Rolling RMS, healthy (top) against hydraulic blockage (bottom). Same level, same shape, same step — nothing a vibration-only model can key on.



The same story in PCA: the two classes occupy one overlapping cloud.

Notes
ENG
IN

Transitions Animations Slide Show Record Review View Help Present in Teams
Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

FAULT BEHAVIOUR

SIEMENS

What each fault does to the pump, and to the signal

State	What happens in the pump	What it does to the vibration signal	How separable
Healthy	Pump running at its design duty point.	Stable RMS, no impulsive content — the baseline everything else is measured against.	Reference
Cavitation	Local pressure drops below vapour pressure, so bubbles form and collapse inside the pump.	Highest sustained rolling-RMS, and strongly impulsive: elevated crest factor, high kurtosis, broad spectral entropy, burst-like segments, more high-frequency content.	Easiest — most distinct
Suction blockage	Restriction at the inlet: clogged strainer, part-closed suction valve, debris, air ingress.	Strong rolling-RMS fluctuation and high variability, strong energy modulation, large transient peaks and irregular bursts.	Clear, but unstable
Hydraulic blockage	Restriction on the discharge side: part-closed valve, downstream obstruction, fouling.	Only a moderate RMS rise over healthy and a slightly broader spread — not impulsive, not unstable. Sits on top of the healthy distribution.	Not separable on vibration

Read the last column as the design decision: cavitation and suction blockage each leave a signature a small model can learn, so they are in. Hydraulic blockage leaves almost none, so it is out until another sensor channel gives it one.

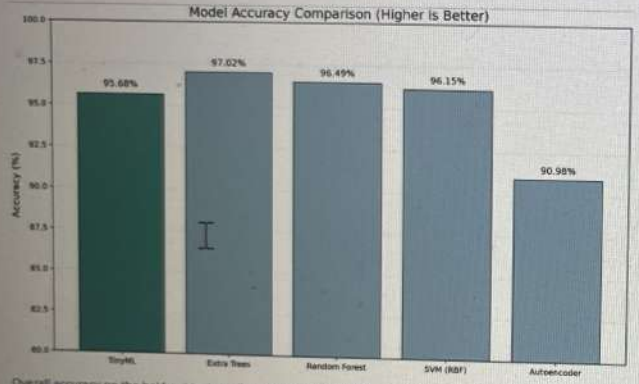
Notes

Transitions Animations Slide Show Record Review View Help
Present in Teams
the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

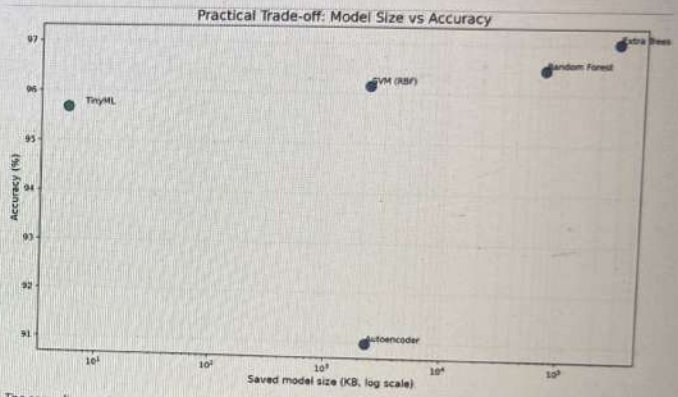
MODEL COMPARISON · 5,900-SAMPLE TEST SET

SIEMENS

Five models were trained on the same 76 features



Overall accuracy on the held-out test set. Extra Trees leads; TinyML sits 1.34 points behind it.



The same five models against saved model size, on a log axis. Note how far left TinyML sits.

The trade-off in one line: Extra Trees is the most accurate at 97.02 %, but its saved model runs to hundreds of megabytes. TinyML gives up 1.34 points to reach 95.68 % in 8.94 KB on the board — four to five orders of magnitude smaller. None of the other four fit on a microcontroller at all.

Notes
ENG IN

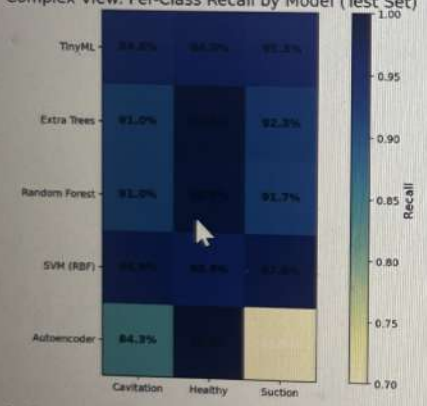
Pump_Fault_TinyML_ESP32_v3.pptx - Protected View C1 - Restricted* • Saved to this PC
Transitions Animations Slide Show Record Review View Help
Present in Teams
m the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

PER-CLASS PERFORMANCE

SIEMENS

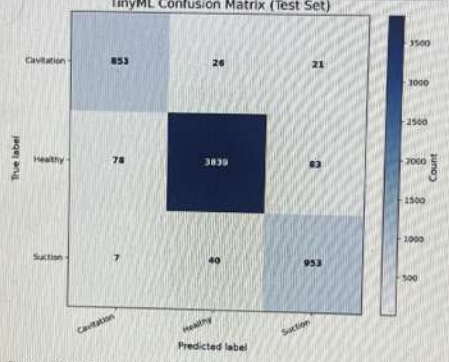
Headline accuracy hides where the models actually differ

Complex View: Per-Class Recall by Model (Test Set)



Recall per class, per model. Read the columns, not the rows.

TinyML Confusion Matrix (Test Set)



TinyML on the full test set: 5,645 of 5,900 correct.

WHAT THE COLUMNS SAY

- **Healthy is 4,000 of 5,900 test samples** — 68 %. A model that simply favours Healthy scores well overall.
- **On the two fault classes TinyML beats both tree ensembles:** 94.8 % cavitation and 95.3 % suction, against 91.0 / 92.3 for Extra Trees and 91.0 / 91.7 for Random Forest.
- **Only SVM (RBF) detects faults better** — 96.9 % and 97.6 % — and that model is megabytes, not kilobytes.
- **The autoencoder is the outlier:** 99.8 % on Healthy but 61.9 % on suction blockage. It mostly learned to say "normal".

So the ranking flips depending on what you measure: Extra Trees wins on overall accuracy, TinyML wins on fault detection among the deployable-sized models, and it is the most balanced of the five across the three classes.

Notes
ENG

Transitions Animations Slide Show Record Review View Help
Present in Teams
e Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

VERIFIED METRICS · ESP32-S3

SIEMENS

Size, speed and memory measured on the board

95.68 %
OFFLINE, FULL TEST SET
5,900 samples

8.94 KB
MODEL ON FLASH
21.87 KB deployed in total

59.02 ms
INFERENCE PER SAMPLE
min 58 · max 64 ms

29.58 KB
PEAK RAM USED
0.36 % of the 8.1 MB heap

SPACE — WHAT SITS ON THE BOARD

File	Bytes	Size
tiny_ml_model_mp.json	9,156	8.94 KB
main.py	12,948	12.64 KB
input_features.csv	153	0.15 KB
boot.py	139	0.14 KB
Deployed total	**22,396**	**21.87 KB**

TIME — BATCH RUN, 100 SAMPLES

Stage	Measured
Model load, once at startup	34 ms
Inference, average	59.02 ms
Inference, min / max	58 / 64 ms
100 samples, end to end	6,907 ms
Throughput	14.48 /s

MEMORY — MICROPYTHON HEAP

Stage	Measured
Total heap available	8,126.50 KB
Free at start	8,112.91 KB
Model resident in RAM	14.14 KB
Peak growth during run	29.58 KB
Peak share of heap	0.36 %

HOW THESE WERE MEASURED

Printed by the board itself: ESP32-S3 at 240 MHz, 8 MB PSRAM, MicroPython v1.29.0, ulab not available. MicroPython does not expose CPU %, so latency and throughput are the CPU indicators. Throughput covers the whole loop (69.1 ms per sample incl. CSV parsing); 59.02 ms is inference alone.

SHRINKING THE MODEL COST ALMOST NOTHING

95.68 % offline on 5,900 samples, 96.0 % on the 100-point batch run and 96.2 % on the 1,000-point stream. Three independent measurements inside a point of each other — converting the trained model to the 8.94 KB on-device form did not measurably hurt it.

Notes
ENG IN

Transitions Animations Slide Show Record Review View Help Present in Teams
e Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

STREAMING · PROVEN ON ESP32-S3

SIEMENS

Streaming works: 1,000 points classified live on the board

96.2 %
STREAM ACCURACY
962 correct of 1,000

54.47 ms
INFERENCE PER POINT
min 54 · max 60 ms

1.81 /s
END-TO-END THROUGHPUT
551,308 ms for 1,000 points

36.90 KB
RESULT LOG WRITTEN
on-device, during the run

```
=== Stream Summary ===
Points processed: 1000
Labeled points: 1000
Correct: 962
Incorrect: 38
Accuracy: 0.962
End-to-end stream time: 551308 ms
Overall throughput: 1.81 points/sec
Per-point latency (ms): avg=54.47 min=54 max=60
Minimum free RAM during stream: 8280144 bytes (8086.08 KB)
Result log size: 37784 bytes (36.90 KB)
```

The board's own stream summary, printed at the end of the run.

WHAT THE STREAMING BUILD COSTS

File	Bytes	Size
main.py (streaming)	19,828	19.36 KB
tiny_ml_model_mp.json	9,156	8.94 KB
input_features.csv	153	0.15 KB
Code + model deployed	**29,137**	**28.45 KB**
stream_results_log.csv (output)	37,784	36.90 KB

Memory held steady: 32.39 KB allocated before the stream, 37.48 KB after, and free RAM never fell below 8,086.08 KB.

INFERENCE IS NOT THE BOTTLENECK — THE LINK IS

Each point takes 552 ms end to end, but only 54.47 ms of that is the model. The other ~90 % is moving the point over the serial link from the PC. On the MS200 the sensor generates the data where it is classified, so most of that transport cost disappears — the 54 ms of inference is what remains.

Notes
ENG IN

Transitions Animations Slide Show Record Review View Help
Present in Teams
he Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

NEXT STEPS

SIEMENS

From a proven stream to learning on the sensor

01

Move the stream onto the MS200

Streaming is proven on the ESP32 with data pushed from a PC. The MS200 — the sensor that will actually be used — generates the stream itself, which also removes the transport cost that dominates the loop today.

02

On-device learning on the ESP32

The main line of work now: move from inference-only to learning on the device, so the model can adapt to the machine it is mounted on instead of being frozen at training time.

03

Then learning on the MS200 itself

Push the same capability down onto the sensor, so no separate board is needed at the pump — the sensor measures, classifies and adapts on its own.

ALSO ON THE LIST

- **Bring hydraulic blockage back** once temperature and other channels join the accelerometer and give it a signature to learn.
- **Re-test the SVM route** — it is the only model that detects faults better than TinyML, so it is worth asking how small it can be made.

WHY IT MATTERS

Each step removes a dependency. Streaming removed the file. Learning on the ESP32 removes the retraining trip back to a PC. Learning on the MS200 removes the extra board — leaving a sensor that diagnoses the pump it is bolted to.

Where this ends up: a sensor bolted to the pump that measures, classifies and keeps learning on its own — no PC, no gateway and no cloud in the loop.

Notes
68%
ENG
IN
12:08
10/11/20

Pump_Fault_TinyML_ESP32_v3.pptx - Protected View C1 - Restricted* • Saved to this PC
Transitions Animations Slide Show Record Review View Help
Present in Teams
the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View. Enable Editing

NEXT STEPS

SIEMENS

From a proven stream to learning on the sensor

01

Move the stream onto the MS200

Streaming is proven on the ESP32 with data pushed from a PC. The MS200 — the sensor that will actually be used — generates the stream itself, which also removes the transport cost that dominates the loop today.

02

On-device learning on the ESP32

The main line of work now: move from inference-only to learning on the device, so the model can adapt to the machine it is mounted on instead of being frozen at training time.

03

Then learning on the MS200 itself

Push the same capability down onto the sensor, so no separate board is needed at the pump — the sensor measures, classifies and adapts on its own.

ALSO ON THE LIST

- **Bring hydraulic blockage back** once temperature and other channels join the accelerometer and give it a signature to learn.
- **Re-test the SVM route** — it is the only model that detects faults better than TinyML, so it is worth asking how small it can be made.

WHY IT MATTERS

Each step removes a dependency. Streaming removed the file. Learning on the ESP32 removes the retraining trip back to a PC. Learning on the MS200 removes the extra board — leaving a sensor that diagnoses the pump it is bolted to.

Where this ends up: a sensor bolted to the pump that measures, classifies and keeps learning on its own — no PC, no gateway and no cloud in the loop.

Notes
ENG

File C:/Users/z005bnta/Downloads/updates_devansh/MODEL_COMPARISON_REPORT_UPDATED.pdf

Draw | Ask Copilot | 1 of 7

Pump Fault Detection Models

Comprehensive Architecture & Performance Analysis

Model	Accuracy	Size (KB)	Size (MB)
TinyML	95.68%	5.43	0.00530
Traditional Autoencoder	75.39%	8.59	0.00839
TinyML-style Autoencoder	96.08%	1186.83	1.15901

Model 1: TinyML Classifier

Metric	Value
Accuracy	95.68%
Model Size (KB)	5.43
Model Size (MB)	0.00530

Workflow Architecture:

Supervised Classification → Direct class prediction from input features

Network Architecture & Training Configuration:

Input Layer: 76 vibration features
Hidden Layer 1: 20 neurons (ReLU activation)
Hidden Layer 2: 10 neurons (ReLU activation)
Output Layer: 3-class softmax (Healthy/Suction/Cavitation)

Training Configuration:
- Optimizer: Adam (lr=0.005)
- Batch Size: 32
- Epochs: 50
- Loss Function: Categorical Cross-Entropy
- L2 Regularization: 0.001
- Quantization: 32-bit → 8-bit (int8)

Per-Class Performance (Test Set):

Class	Recall	Precision	F1-Score
Cavitation	94.78%	90.94%	0.9282
Healthy	95.97%	98.31%	0.9713
Suction	95.30%	90.16%	0.9266

Search

Metric	Value
Accuracy	75.39%
Model Size (KB)	8.59
Model Size (MB)	0.00839

Workflow Architecture:

Unsupervised Reconstruction + Centroid-based Classification

Network Architecture & Training Configuration:

Encoder Path:
76 features → 20 neurons (ReLU) → 10 neurons (ReLU) [Latent Space]

Decoder Path (Reconstruction):
10 neurons (ReLU) → 20 neurons (ReLU) → 76 features (Linear)

Classification Head (Latent Space):
10-dimensional latent vector → Nearest-Centroid Classifier

Training Configuration:
- Optimizer: Adam (lr=0.005)
- Batch Size: 32
- Epochs: 50
- Loss: MSE (reconstruction)
- L2 Regularization: 0.001

Per-Class Performance (Test Set):

Class	Recall	Precision	F1-Score
Cavitation	56.33%	55.23%	0.5578
Healthy	95.55%	85.87%	0.9045
Suction	11.90%	22.41%	0.1555

Model 3: TinyML-style Autoencoder (Hybrid)

Metric	Value
Accuracy	96.08%
Model Size (KB)	1186.83
Model Size (MB)	1.15901

Workflow Architecture:

Ultra-Deep Hybrid Learning: TinyML Supervision + Autoencoder Feature Learning + Batch Normalization

Network Architecture & Training Configuration:

Encoder Path (Ultra-Deep):
76 → 128 → 96 → 64 → 48 → 32 → 16 → 12 neurons (ReLU + BatchNorm + Dropout)

Decoder Path (Reconstruction - TinyML-inspired):
12 → 16 → 32 → 48 → 64 → 96 → 128 → 76 (ReLU + BatchNorm + Dropout)

Classification Head (TinyML-style Supervised):
12D latent → 64 → 64 → 64 neurons → 3-class softmax (ReLU + BatchNorm + Dropout)

Training Configuration:
- Optimizer: Adam (lr=0.0005, adaptive reduction)
- Batch Size: 16 (smaller for stability)
- Epochs: 200
- Hybrid Loss: 60% Reconstruction MSE + 40% Classification CrossEntropy (TinyML-inspired)
- Batch Normalization: Every layer
- Dropout Rate: 0.3 (aggressive)
- L1/L2 Regularization: L1=0.00005, L2=0.0001
- Framework: TensorFlow/Keras
- Early Stopping: patience=30

Per-Class Performance (Test Set):

Class	Recall	Precision	F1-Score

Search

Model 3: TinyML-style Autoencoder (Hybrid)

Metric	Value
Accuracy	96.08%
Model Size (KB)	1186.83
Model Size (MB)	1.15901

Workflow Architecture:

Ultra-Deep Hybrid Learning: TinyML Supervision + Autoencoder Feature Learning + Batch Normalization

Network Architecture & Training Configuration:

Encoder Path (Ultra-Deep):
76 → 128 → 96 → 64 → 48 → 32 → 16 → 12 neurons (ReLU + BatchNorm + Dropout)

Decoder Path (Reconstruction - TinyML-inspired):
12 → 16 → 32 → 48 → 64 → 96 → 128 → 76 (ReLU + BatchNorm + Dropout)

Classification Head (TinyML-style Supervised):
12D latent → 64 → 64 → 64 neurons → 3-class softmax (ReLU + BatchNorm + Dropout)

Training Configuration:
- Optimizer: Adam (lr=0.0005, adaptive reduction)
- Batch Size: 16 (smaller for stability)
- Epochs: 200
- Hybrid Loss: 60% Reconstruction MSE + 40% Classification CrossEntropy (TinyML-inspired)
- Batch Normalization: Every layer
- Dropout Rate: 0.3 (aggressive)
- L1/L2 Regularization: L1=0.00005, L2=0.0001
- Framework: TensorFlow/Keras
- Early Stopping: patience=30

Per-Class Performance (Test Set):

Class	Recall	Precision	F1-Score

Search